Solution: Unique Number of Occurrences (easy)

Problem Statement

Given an array of integers, determine if the number of times each distinct integer appears in the array is unique.

In other words, the occurrences of each integer in the array should be distinct from the occurrences of every other integer.

Examples:

1. - **Input**: [4, 5, 4, 6, 6, 6]
   - **Expected Output**: true
   - **Justification**: The number 4 appears 2 times, 5 appears 1 time, and 6 appears 3 times. All these occurrences (1, 2, 3) are unique.
2. - **Input**: [7, 8, 8, 9, 9, 9, 10, 10]
   - **Expected Output**: false
   - **Justification**: The number 7 appears 1 time, 8 appears 2 times, 9 appears 3 times, and 10 appears 2 times. The occurrences are not unique since the number 2 appears twice.
3. - **Input**: [11, 12, 13, 14, 14, 15, 15, 15]
   - **Expected Output**: false
   - **Justification**: The number 11 appears 1 time, 12 appears 1 time, 13 appears 1 time, 14 appears 2 times, and 15 appears 3 times. These occurrences are not unique.

Solution

The solution to this problem involves two main steps. First, we count the occurrences of each element in the array using a hashmap. Each element of the array serves as a key, and its frequency as the value.

After we have the frequencies, the second step is to ensure the uniqueness of these frequencies. We achieve this by storing each frequency in a hashset. Since a hashset does not allow duplicates, if we encounter a frequency that's already in the set, it indicates that there are at least two elements with the same frequency, and thus, we return false. If all frequencies are unique, our final result is true.

Here is the step-by-step solution.

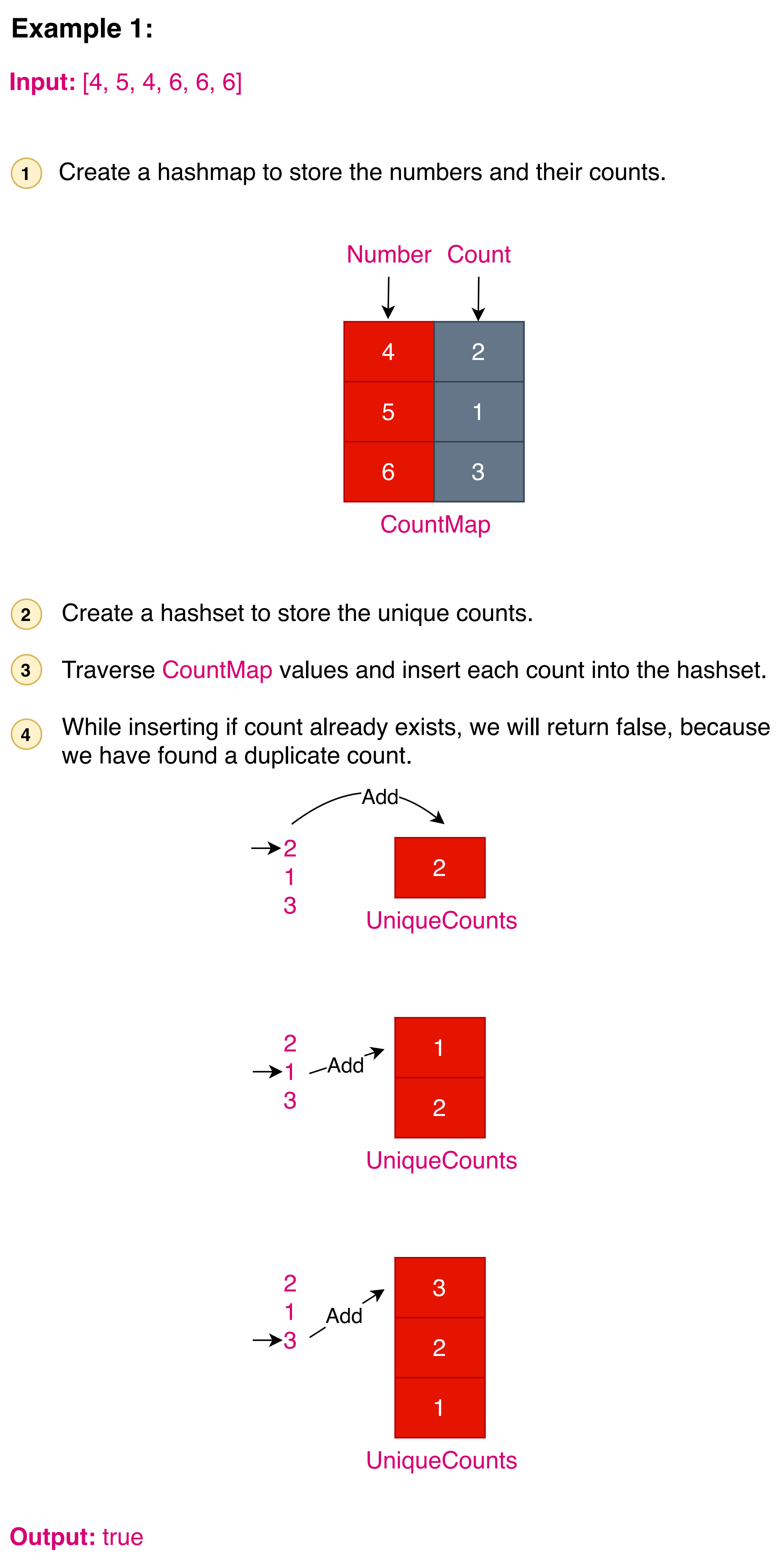
1. **Counting Occurrences**: Start by counting the occurrences of each integer in the array. This can be done using a hashmap where the key is the integer and the value is its count.
2. **Checking Uniqueness**: Once we have the counts, we need to check if these counts are unique. This can be done by inserting each count into a hashset. If at any point, we try to insert a count that already exists in the set, we can conclude that the occurrences are not unique.
3. **Return Result**: If all counts are successfully inserted into the set without any repetition, then the occurrences are unique, and we return true. Otherwise, we return false.

This approach ensures that we determine the uniqueness of occurrences in an efficient manner. By using a hashmap to count occurrences and a hashset to check for uniqueness, we can solve the problem in linear time.

Algorithm Walkthrough: Given the input [4, 5, 4, 6, 6, 6]:

1. Initialize an empty hashmap `countMap`.
2. Traverse the array. For each integer, increase its count in `countMap`.
   - After traversal, `countMap` will be: {4:2, 5:1, 6:3}
3. Initialize an empty hashset `uniqueCounts`.
4. Traverse the values of `countMap`. For each count, try to insert it into `uniqueCounts`.
   - Insert 2 (from integer 4) into `uniqueCounts`.
   - Insert 1 (from integer 5) into `uniqueCounts`.
   - Insert 3 (from integer 6) into `uniqueCounts`.
5. Since all counts were inserted without repetition, return true.

Here is the visual representation of the algorithm:



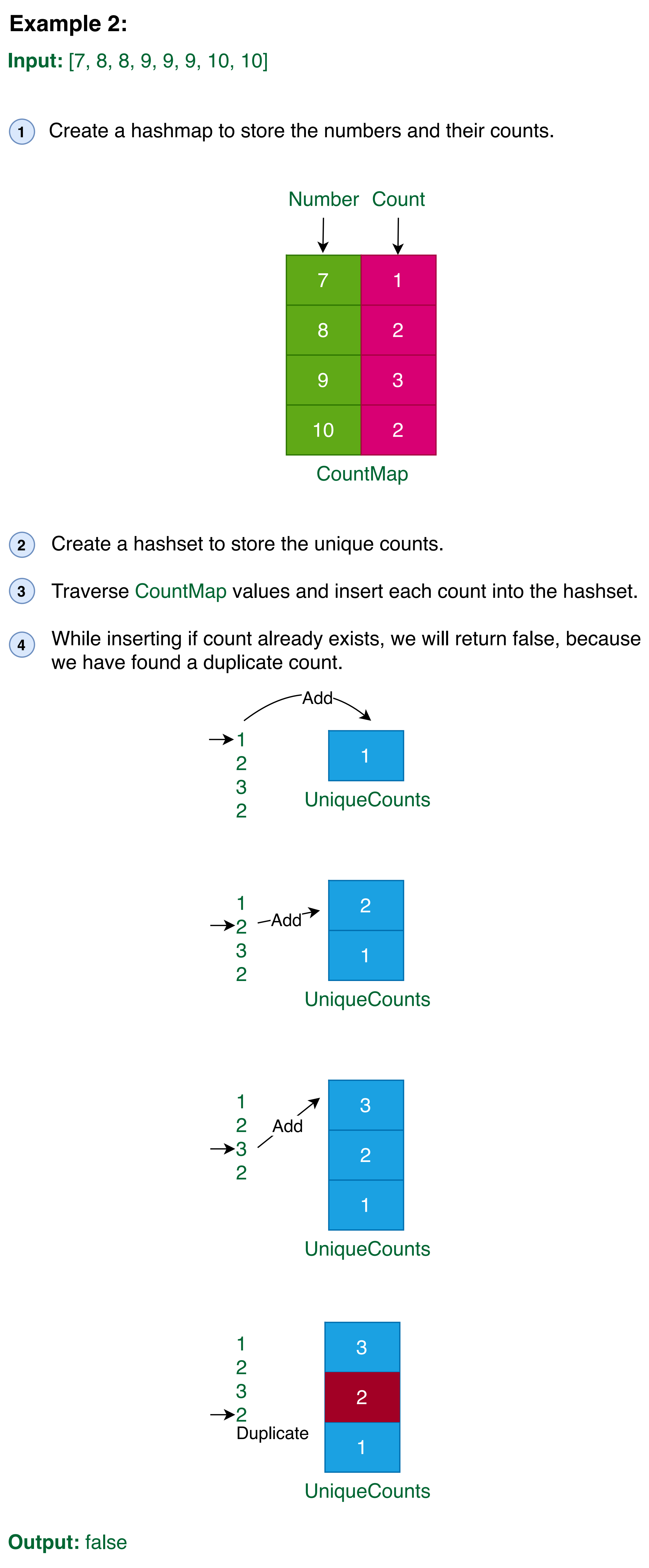
Example 1:

Input: [4, 5, 4, 6, 6, 6]

1. Create a hashmap to store the numbers and their counts.
2. Create a hashset to store the unique counts.
3. Traverse CountMap values and insert each count into the hashset.
4. While inserting if count already exists, we will return false, because we have found a duplicate count.

Output: true

Unique Number of Occurrences Ex1.



Example 2:

Input: [7, 8, 8, 9, 9, 9, 10, 10]

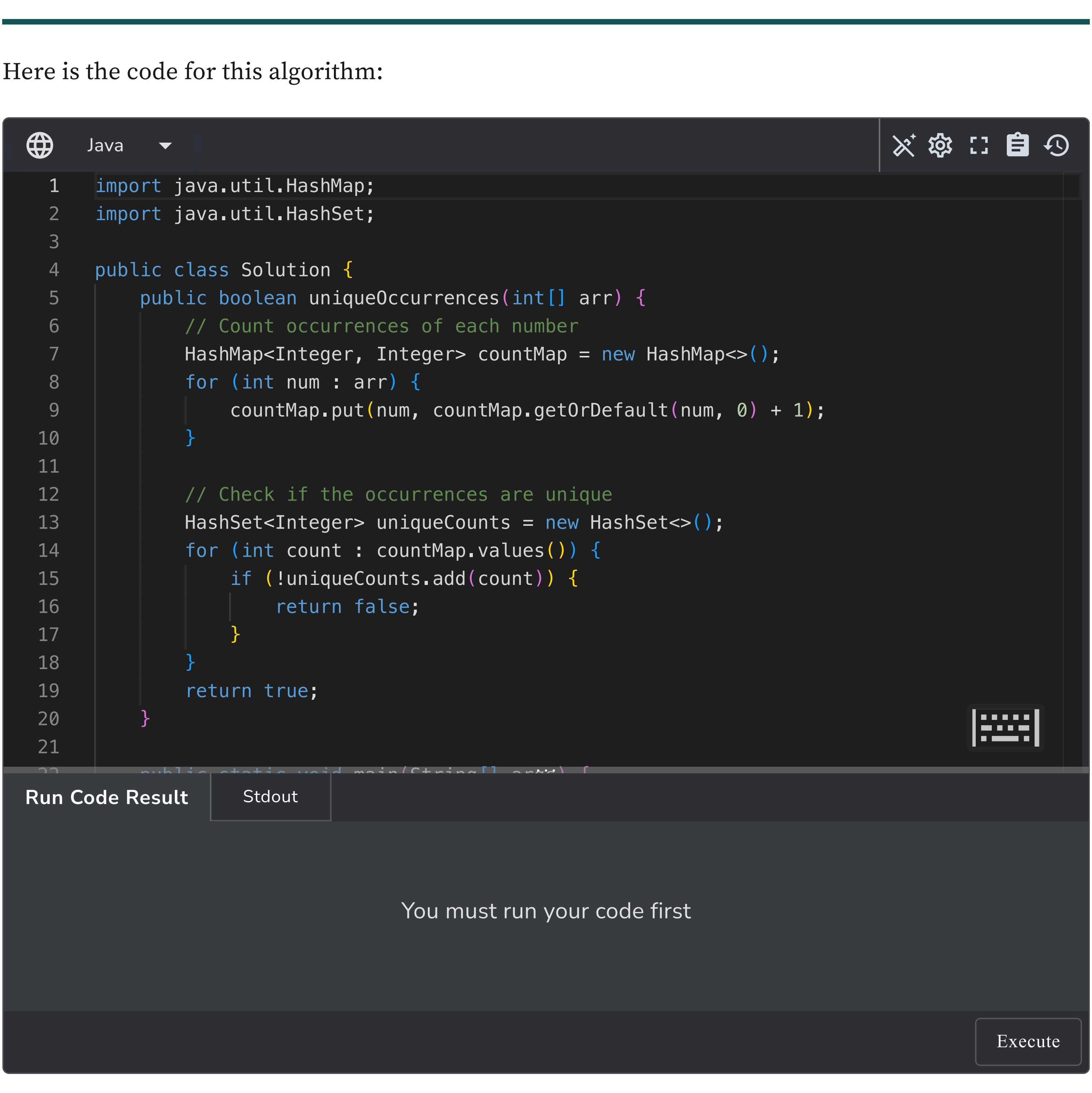
1. Create a hashmap to store the numbers and their counts.
2. Create a hashset to store the unique counts.
3. Traverse CountMap values and insert each count into the hashset.
4. While inserting if count already exists, we will return false, because we have found a duplicate count.

Output: false

Unique Number of Occurrences Ex2.

Code

Here is the code for this algorithm:



```java
import java.util.HashMap;
import java.util.HashSet;

public class Solution {
    public boolean uniqueOccurrences(int[] arr) {
        // Count occurrences of each number
        HashMap<Integer, Integer> countMap = new HashMap<>();
        for (int num : arr) {
            countMap.put(num, countMap.getOrDefault(num, 0) + 1);
        }

        // Check if the occurrences are unique
        HashSet<Integer> uniqueCounts = new HashSet<>();
        for (int count : countMap.values()) {
            if (!uniqueCounts.add(count)) {
                return false;
            }
        }
        return true;
    }

```

You must run your code first

Time Complexity:

1. **Counting Occurrences:** We iterate through the entire array to count the occurrences of each number. This operation takes (O(n)) time, where (n) is the length of the array.
2. **Checking Unique Occurrences:** After counting, we iterate through the count map (or dictionary) to check if the occurrences are unique. In the worst case, every number in the array is unique, so the size of the count map is (n). Thus, this operation also takes (O(n)) time.

Combining the two steps, the overall time complexity is (O(n) + O(n) = O(n)).

Space Complexity:

1. **Count Map:** We use a hashmap (or dictionary) to store the count of occurrences of each number. In the worst case, if all numbers in the array are unique, the size of the count map is (n). So, the space complexity for the count map is (O(n)).
2. **Unique Counts Set**: We use a hashset to store unique counts. In the worst case, if all numbers in the array have a unique count (which is highly unlikely and would mean that each number repeats a unique number of times), the size of the set is (n). So, the space complexity for the unique counts set is (O(n)).

Combining the two data structures, the overall space complexity is (O(n) + O(n) = O(n)).

← Previous
Problem 3: Unique Number Of Occurrences (Easy)

Next →
Problem 4: Longest Substring Without Repeating Characters (Medium)

☐ Mark as Completed